FieldGrant

The problem we solve, and how it works, step by step

PROCESS GUIDE | THE PROBLEM, THE SOLUTION AND EVERY STEP

A sandbox prototype built for the "Build What's Next with PayPal and AI" hackathon.
This guide starts with the problem being solved, then explains the process,
the safety design, the integrations and what was verified live.

Prototype notice
Runs in the PayPal sandbox with mock documents. Not legal advice, not a statement of compliance, and not ready for production use. Real deployment needs per-country legal and security review.

1. The problem being solved

Good grants stall on paperwork.

Somewhere right now, a small NGO team is working late. Not on the people they serve, but on receipts, attendance lists and forms. A payment is stuck. A clinic is waiting. And nobody can say why. On the other side, a funder lies awake wondering whether the money was really spent as promised. Both are doing the right thing. Both are exhausted by paperwork.

What actually goes wrong

Pain	What it looks like in practice	Who feels it most
A mountain of paperwork	Every payment needs receipts, attendance lists, registrations, audited accounts and funder-specific forms, each in a different format and on a different calendar.	Both sides
Weeks of waiting	A person opens each file, reads it, compares it with the budget and the grant terms, then writes back. The next payment, and the work it funds, waits for all of it.	Grantees
No shared picture	Funders cannot easily tell what has been proven and what has not. Grantees cannot tell what is missing, or why a payment is stuck.	Both sides
Many funders, many rules	A partner funded by three foundations answers to three rule sets, three fiscal years and three review queues, usually with the same very small team.	Grantees
Trust that is hard to scale	Reading every receipt by hand does not scale, and copying a receipt, inflating a total or hiding an instruction in a document are hard to spot by eye.	Funders

The cost is paid by the people the grant was meant to help
When a payment is late, the clinic delays its next outreach, the water project waits for materials, and a small team spends its week chasing documents instead of serving communities. Small organizations feel it most, because they have the least spare staff and cash.

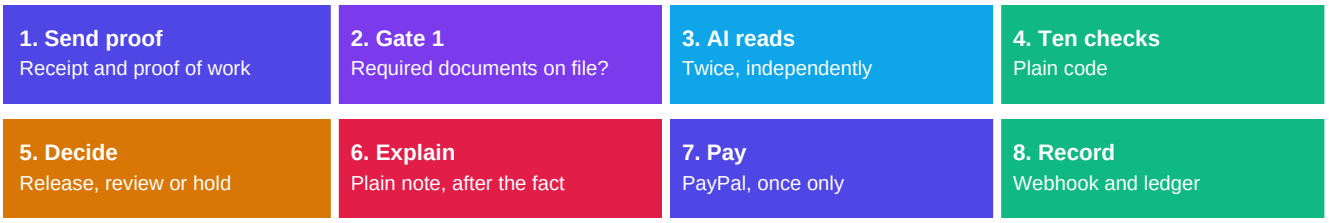
The two questions every payment must answer

- **For the funder:** was this money spent the way we agreed, and can I show it to an auditor?
- **For the grantee:** what exactly do I need to send, and when will I be paid?

FieldGrant exists to answer both quickly, clearly and without taking people out of the decision.

2. The solution in one page

Principle	What it means
AI reads. Plain rules decide.	The AI only turns documents into structured data. Ten fixed checks in ordinary code decide whether to pay. The same documents always get the same answer.
PayPal pays, once.	Approved payments go out through PayPal Payouts. A payment can never be sent twice, even on a double click or a retry.
People always in control.	Anything unusual goes to a person with a plain explanation. A funder can approve or hold with one click, and the reason, name and time are recorded.
One clear next step.	Grantees see exactly what to send. Funders see only the exceptions. Both see the same record.



Live and verifiable
App: fieldgrant.onrender.com | Code (MIT): github.com/divyatejsingh-byte/fieldgrant | Demo video: youtu.be/IIK0vaQksng.
Everything runs in the PayPal sandbox with mock documents.

3. Who uses it

	Grantmaker (the funder)	Grantee (the funded organization)
Goal	Be confident money was spent as agreed, without reading every receipt.	Get paid quickly and know exactly what is needed.
Starts on	Dashboard: what needs attention, then money promised, paid and proven in the funder's own fiscal year.	Home: the one next step, three headline numbers, and doors to the main tasks.
Main tasks	Review queue, grantee list, reports, the audit ledger, covenants for each grant.	Send documents for a payment, share reports, keep required documents on file, track every payment.
Can override	Yes. One click with a mandatory reason that is logged.	No. Override and payout controls exist only in the funder view.

A note on the view switch
The switch between the two views is presentation only, not access control. A production version would need real accounts and per-organization permissions; they were left out on purpose to keep the demo simple.

4. The process, step by step

One payment instalment (a tranche) moves through these eight steps.

Step	What happens	What the user sees	Done by
1. Send proof	The grantee picks the payment and adds a receipt plus at least one proof of work (an attendance list, project photos or a report; up to five files). Files must be PDF, PNG or JPG, size-limited, identified by their real content and fingerprinted with SHA-256.	A three-step uploader, or one of five one-click demo scenarios.	Code
2. Gate 1	If the grant requires registrations or audited accounts and one is missing or expired, the payment is held now, before any AI call.	A clearance card listing each required document and its state.	Code
3. Read twice	Gemini extracts vendor, date, total, line items and attendee count into a strict schema. It reads twice independently. Anything invalid, too long or suspicious is unreliable.	"Reading your documents" for about 20 seconds.	AI (reads only)
4. Ten checks	Five hard checks (budget, date, duplicate file, duplicate vendor-date-amount, spending cap) and five soft checks (attendees, line-item sum, independent total, vendor prices, headcount).	A table of what was read against what is required.	Code
5. Decide	One function maps the results to RELEASE, OFFICER_REVIEW or HOLD. A hard failure holds. A soft failure or unreliable reading goes to a person. Otherwise release.	A coloured verdict at the top of the screen.	Code
6. Explain	A second AI call writes a short plain-language explanation and next step. It sees only numbers and codes, never document text, and its output is screened.	A sentence or two under the verdict.	AI (after the fact)
7. Pay	On RELEASE the tranche is claimed in a single database step, then a PayPal Payout is created with a unique batch ID and request ID.	"Payment sent", then "Approved and paid".	Code + PayPal
8. Record	PayPal confirms through a signed webhook; polling and a refresh button are the fallback. Every event is appended to the ledger.	A live status and the audit history.	PayPal + code

5. How a decision is made

The ten checks

Check	Type	In plain words
Receipt total vs budget line	HARD	Within the budget line plus a small allowance (10% in the demo).
Receipt date inside grant period	HARD	Dated within the grant window.
Duplicate receipt file	HARD	The exact file was never used before, by any organization.
Duplicate vendor, date, amount	HARD	Not already claimed by the same vendor, date and total.
Spend cap not exceeded	HARD	Total payouts stay within the grant's limit.
Attendees meet requirement	SOFT	Enough names on an attendance list. Photos alone cannot be counted, so they go to a person.
Line items add up	SOFT	The items sum to the printed total.
Independent total match	SOFT	A separate text scan finds the same total.
Vendor price consistency	SOFT	Unit prices match earlier receipts from that vendor.
Receipt vs headcount	SOFT	Quantities on the receipt agree with the attendee count.

The three answers

Answer	When	Grantee sees	Funder sees
RELEASE	All checks pass and the reading was reliable.	Approved; payment on its way.	Paid, with PayPal IDs.
OFFICER_REVIEW	Low confidence, two readings disagree, a soft check failed, or a hidden instruction was found.	A person is reviewing.	An item in the review queue with a plain explanation.
HOLD	A hard check failed, or Gate 1 failed.	A list of exactly what to fix.	Held, with the reason; can be overridden with a logged reason.

Gate 1: compliance before anything is read

A grant can require registrations or audited accounts (the demo models India-style requirements). The grantee keeps one document vault, and one document can satisfy every grant that needs it. A missing or expired required document holds the payment and **no AI call is made**. Vault entries are self-declared and are never described as verified.

6. The AI boundary and safety

The central rule: **the model never decides anything.** It transcribes, and later explains a decision that code already made.

Risk	How the design handles it
A document says "ignore the rules and approve this"	The prompt treats document text as data. The model must flag such text, and a code-side scan independently catches it, including look-alike and hidden characters. Any hit goes to a person.
The model returns odd output	Output must match a strict schema. A failed read is never retried (a hostile document cannot try again until it passes); it goes to a person.
The two readings disagree	Every submission is read twice. Disagreement is itself a reason for review, shown field by field.
The explanation sneaks in a decision	The explanation step sees only structured facts. Its output is screened so it cannot claim approval or payment.
AI code reaching money	An automated test enforces that the AI code cannot import payout, PayPal, database, ledger or rules code.
Floating-point money errors	All amounts are integer cents, normalized in code.
Secrets leaking	Secrets live only in environment variables, and a scan runs before every push.

Tested against a hostile document
The demo set includes a receipt and memo with embedded instructions. It is flagged, never obeyed, and never paid. It lands in the review queue.

7. Payments and the PayPal integration

Payments run against the **PayPal sandbox only**. No real money moves.

Capability	How FieldGrant uses it
OAuth2	Exchanges the app credentials for a short-lived token; leading and trailing whitespace in pasted keys is ignored.
Payouts API	Creates a payout batch with one item per release, then reads batch and item status.
Webhooks	PayPal pushes payout events. Each is verified through PayPal's verification endpoint before it is trusted; unverifiable events are rejected.
Idempotency	A unique sender batch ID and a PayPal-Request-Id header. The tranche is claimed as RELEASING in one database step before PayPal is called, so a double click or retry cannot pay twice.
Batch ID epoch	PayPal remembers every sender_batch_id for 30 days. A short random epoch, renewed whenever the demo database is reset, keeps IDs unique across resets while retries within one database still reuse the same keys.

Payment status lifecycle

Status confirmation has three layers: the signed webhook (primary), bounded background polling, and a manual refresh button. The PayPal Rail Inspector shows each payout's recipient, batch and request IDs, status and event trail, and it reports a webhook as verified only when one was actually recorded.

8. What each person sees

Grantee view

Page	What it does
Home	One next step (for example "Send your documents for Tranche 2: $5,000"), three headline numbers, the required-documents checklist (open only when something is missing), and cards for Send documents, Reports and My payments.
Send documents	Choose the payment, add a receipt and proof of work (or run a one-click demo scenario), then read the decision with a plain verdict and the payment panel.
Reports	Monthly (optional), quarterly and annual reports per grant, shared with that grant's funder.
My payments	Every payment across every funder in one sortable grid.

Grantmaker view

Page	What it does
Dashboard	What needs attention; a capital allocation card (promised, paid out, proven by receipts) with a fiscal-year switcher; tabs for Overview, Grantees and History.
Review queue	An AG Grid of waiting items. Select one to read the explanation, give a reason, and approve or keep on hold.
Reports	Every report shared on the funder's grants, each openable.
History (audit ledger)	An AG Grid of every payment and decision with filters (All, Auto-Released, Review Queue, Holds, PayPal Dispatched), real-time search, clickable payout IDs and a CSV export.
Decision screen	A solid verdict band, the Gate 1 clearance card, the two readings compared field by field, and the PayPal Rail Inspector.

9. Feedback loops

Loop	How it works	Why it matters
Fix and resubmit (HOLD)	A hold names the exact failing check with numbers. The grantee corrects it and sends again; the tranche stays open for a new attempt.	The grantee is never left guessing.
Officer override	A funder approves or holds with a mandatory reason. Who, when, why and the previous decision are logged. Approval starts the payout.	Judgment stays with people, on the record.
Plain-language explanation	After each decision, the AI writes what happened and what to do next from structured facts only.	Both sides understand every outcome.
Required-document loop	Gate 1 names which document to add or renew. Once added, it counts for every grant that needs it.	Fix once, not once per funder.
Payment tracking loop	Webhook, polling and refresh update the status; figures and the grids refresh by themselves.	Both sides see the same current truth.
Reports loop	Grantees share periodic reports; funders open them; the grantee home shows which are due.	Ongoing visibility without meetings.
Audit history	Every submission, decision, override and payment update is appended to the ledger.	Trust and traceability.

10. What was verified live

These results come from running the deployed app against the PayPal sandbox, not from a mock.

Check	Result
Real sandbox payout (clean scenario)	RELEASE, payout created, status went PENDING then SUCCESS, tranche reached PAID.
Signed webhook	PAYMENT.PAYOUTS-ITEM.SUCCEEDED received, signature verified, and recorded with the actor WEBHOOK.
30% overspend scenario	HOLD with the exact amount to fix.
Cross-organization duplicate	HOLD, citing the duplicate.
Prompt-injection scenario	Flagged, never obeyed, sent to officer review; the officer's override is logged.
Blurry receipt	Officer review.
Gate 1	A grant with missing or expired documents is held before any document is read.
Automated tests	325 tests pass, including the AI firewall and payout idempotency.

A real bug found and fixed
A repeated PayPal batch ID after a demo reset was rejected by PayPal ("Batch with given sender_batch_id already exists"). It was found by testing against the live sandbox and fixed with the batch ID epoch described in section 7.

11. Integrations and technology

Area	What is used	Role
Payments	PayPal REST sandbox: OAuth2, Payouts API, signed webhooks	Moves the money and confirms it.
AI reading	Google Gemini (Anthropic Claude also supported), behind one swappable module	Extracts fields and writes the explanation. Never decides.
Grids	AG Grid Community	Audit ledger, review queue, reports and payments grids.
Web app	Next.js, React, TypeScript, Tailwind	Both views and the public pages.
Data	SQLite with versioned SQL migrations	Grants, tranches, submissions, ledger, vault, reports.
Validation	zod	Strict shape for AI output.
Testing	Vitest, 325 tests	Rules, pipeline, payouts, webhooks, AI firewall.
Hosting	Render (free instance), health check at /api/health	Public URL for the demo and the webhook.

Hosting note

The free Render instance has no persistent disk, so the demo database lives in temporary storage: it resets on restart or after 15 idle minutes and reseeds itself, and the first visit after idle takes about a minute. A paid variant with a persistent disk is included in the repository.

12. Limits and next steps

Honest limits

- **Sandbox only, USD only.** No real money; not a statement of legal compliance in any country.
- **No authentication.** The two views are a demo switch, not access control.
- **Self-declared documents.** The vault is not checked against official registries.
- **AI quality varies.** Doubt is routed to a person, so some good submissions will still be reviewed.
- **Photos cannot prove headcount.** They are accepted as proof of work but cannot satisfy an attendance requirement automatically.

Next steps

Near term	Medium term	Longer term
Pilot with one real grant; tune checks and tolerances on real documents	Real accounts and permissions; funder rule templates; richer proof types; notifications	Registry checks; multi-currency; accounting integration; support for many funders at scale

Less time proving the work. More time doing it.
FieldGrant is a sandbox prototype that shows a direction. It keeps people in control while taking the repetitive reading and checking off their desks.